



Programovacie jazyky

Polymorfizmus, typové triedy a algebraické dátové typy

RNDr. Richard Ostertág, PhD. (ostertag@dcs.fmph.uniba.sk)

7. apríla 2018

Katedra informatiky, Fakulta matematiky, fyziky a informatiky
Univerzita Komenského, Bratislava

Polymorfizmus

Parametrický polymorfizmus

Definícia

Parametrický polymorfizmus, umožňuje pri definícii funkcie použiť typové premenné namiesto konkrétnych typov. Takéto funkcie potom môžu byť podľa potreby inštancované pre konkrétne typy. Všetky inštancie zdieľajú rovnakú implementáciu (kód), čiže vykazujú *rovnaké správanie*.

Príklady

- Java: generické funkcie
- C++: šablóny
- Haskell: polymorfické funkcie

Parametrický polymorfizmus

```
1 map :: (a -> b) -> [a] -> [b]
2 map _ []      = []
3 map f (x:xs) = f x : map f xs
```

```
1 length :: [a] -> Int
2 length [] = 0
3 length (_:xs) = 1 + length xs
```

Definícia

Ad-hoc polymorfizmus, umožňuje *rôzne správanie* pre rôzne inštancie polymorfickej funkcie. Pre rôzne inštancie typovej premennej existujú jej rôzne implementácie.

Príklady

- Java: preťaženie (overloading) metód, operátorov
- C++: preťaženie funkcií, operátorov
- Haskell: typové triedy

Typové triedy

```
1 class Comparable a where
2   eq  :: a -> a -> Bool
3   neq :: a -> a -> Bool
4   lt  :: a -> a -> Bool
5   cmp :: a -> a -> Int
```

Typy ako inštancie typových tried

```
1 instance Comparable Int where
```

```
2   eq  x y = x == y
```

```
3   neq x y = x /= y
```

```
4   lt  x y = x < y
```

```
5   cmp x y = x - y
```

`(1::Int) 'neq' 2` $\rightsquigarrow$ True

```
1 instance Comparable Bool where
```

```
2   eq  x y = x == y
```

```
3   neq x y = x /= y
```

```
4   lt  False True  = True
```

```
5   lt  _      _    = False
```

```
6   cmp False False = 0
```

```
7   cmp False True  = -1
```

```
8   cmp True  False = 1
```

```
9   cmp True  True  = 0
```

`True 'cmp' False` $\rightsquigarrow$ 1

Typové triedy – preddefinovaná implementácia

```
1  class Comparable a where
2    {-# MINIMAL (eq|neq),(lt|cmp) #-}
3    eq  :: a -> a -> Bool
4    eq x y = not (x 'neq' y)
5    neq :: a -> a -> Bool
6    neq x y = not (x 'eq' y)
7    lt  :: a -> a -> Bool
8    lt x y = cmp x y < 0
9    cmp :: a -> a -> Int
10   cmp x y | x 'eq' y  = 0
11           | x 'lt' y  = -1
12           | otherwise = 1
```

- pragma MINIMAL musí byť odsadená v bloku typovej triedy
- „|“ predstavuje „alebo“ a „&“ predstavuje „a zároveň“
- „&“ ma vyššiu prioritu ako „|“

Inšancie využívajúce preddefinovanú implementáciu

Preddefinovaná implementácia výrazne zjednodušuje implementáciu nových inšancií:

```
1 instance Comparable Int where
2   eq x y = x == y
3   cmp x y = x - y
```

```
1 instance Comparable Bool where
2   eq x y = x == y
3   lt False True = True
4   lt _ _ = False
```

Informácie o typových triedach

Po zadaní „:i **Comparable**“ GHCi vypíše rozhranie typovej triedy, minimálnu požadovanú implementáciu a jednotlivé inštancie:

```
1 class Comparable a where
2   eq :: a -> a -> Bool
3   neq :: a -> a -> Bool
4   lt :: a -> a -> Bool
5   cmp :: a -> a -> Int
6   {-# MINIMAL (eq|neq),(lt|cmp) #-}
7
8 instance Comparable Bool
9 instance Comparable Int
```

Štandardné typové triedy

Typová trieda Eq

```
1 class Eq a where
2   (==) :: a -> a -> Bool
3   (/=) :: a -> a -> Bool
4   {-# MINIMAL (==) | (/=) #-}
5
6 instance (Eq b, Eq a) => Eq (Either a b)
7 instance Eq a => Eq [a]
8 instance (Eq a, Eq b) => Eq (a, b)
9 instance Eq ()
10 instance Eq a => Eq (Maybe a)
```

Okrem vyššie uvedených inštancií sú inštanciou tejto typovej triedy aj nasledovné typy: Word, Ordering, Int, Float, Double, Char, Bool, Integer a n -tice až pre $n = 15$.

Typová trieda Ord

```
1  class Eq a => Ord a where
2    compare :: a -> a -> Ordering
3    (<) :: a -> a -> Bool
4    (<=) :: a -> a -> Bool
5    (>) :: a -> a -> Bool
6    (>=) :: a -> a -> Bool
7    max :: a -> a -> a
8    min :: a -> a -> a
9    {-# MINIMAL compare | (<=) #-}
10
11 instance (Ord b, Ord a) => Ord (Either a b)
12 instance Ord a => Ord [a]
13 instance (Ord a, Ord b) => Ord (a, b)
14 instance Ord ()
15 instance Ord a => Ord (Maybe a)
```

Okrem vyššie uvedených inštancií sú inštanciou tejto typovej triedy aj nasledovné typy: Word, Ordering, Int, Float, Double, Char, Bool, Integer a n -tice až pre $n = 15$.

Typová trieda Enum

```
1  class Enum a where
2    succ :: a -> a
3    pred :: a -> a
4    toEnum :: Int -> a
5    fromEnum :: a -> Int
6    enumFrom :: a -> [a]
7    enumFromThen :: a -> a -> [a]
8    enumFromTo :: a -> a -> [a]
9    enumFromThenTo :: a -> a -> a -> [a]
10   {-# MINIMAL toEnum, fromEnum #-}
11
12  instance Enum Word
13  instance Enum Ordering
14  instance Enum Integer
15  instance Enum Int
16  instance Enum Char
17  instance Enum Bool
18  instance Enum ()
19  instance Enum Float
20  instance Enum Double
```

Typová trieda Bounded

```
1 class Bounded a where
2   minBound :: a
3   maxBound :: a
4   {-# MINIMAL minBound, maxBound #-}
5
6 instance (Bounded a, Bounded b) => Bounded (a, b)
7 instance Bounded () -- Defined in 'GHC.Enum'
```

Okrem vyššie uvedených inštancií sú inštanciou tejto typovej triedy aj nasledovné typy: Word, Ordering, Int, Char, Bool, a n -tice až pre $n = 15$.

Typová trieda Num

```
1  class Num a where
2    (+) :: a -> a -> a
3    (-) :: a -> a -> a
4    (*) :: a -> a -> a
5    negate :: a -> a
6    abs :: a -> a
7    signum :: a -> a
8    fromInteger :: Integer -> a
9    {-# MINIMAL (+),(*),abs,signum,fromInteger,(negate | (-)) #-}
10   x - y      = x + negate y
11   negate x = 0 - x -- hmm ..., why not: (fromInteger 0) - x
12
13  instance Num Word
14  instance Num Integer
15  instance Num Int
16  instance Num Float
17  instance Num Double
```


Typová trieda Show

```
1 class Show a where
2   showsPrec :: Int -> a -> ShowS
3   show :: a -> String -- <- povieme si iba o tejto
4   showList :: [a] -> ShowS
5   {-# MINIMAL showsPrec | show #-}
6
7 instance Show a => Show [a] -- Defined in 'GHC.Show'
8 instance Show a => Show (Maybe a) -- Defined in 'GHC.Show'
9 instance (Show b, Show a) => Show (Either a b)
10 instance (Show a, Show b) => Show (a, b) -- Defined in 'GHC.Show'
11 instance Show () -- Defined in 'GHC.Show'
```

Okrem vyššie uvedených inštancií sú inštanciou tejto typovej triedy aj nasledovné typy: Word, Ordering, Int, Float, Double, Char, Bool, Integer a n -tice až pre $n = 15$.

Algebraické datové typy

Záznamy (records, product type)

n-tice (tuples), nie polia, ani zoznamy

```
1 data Student = MkSt String String Int String
2
3 st :: Student
4 st = MkSt "Janko" "Hraško" 4 "šikovný"
```

- **Student** – nový typ (musí začínať veľkým písmenom)
- **MkSt** – dátový konštruktor (musí začínať veľkým písmenom)
 - je to vlastne funkcia:
MkSt::String->String->Int->String->Student
ktorá z jednotlivých zložiek záznamu vytvorí jeho inštanciu
 - môže sa volať rovnako ako typ (t.j. **Student**)

Záznamy – extrakcia položiek

```
1  getMeno :: Student -> String
2  getMeno (MkSt meno _priezvisko _rocnik _poznanka) = meno
3
4  getPriezviko :: Student -> String
5  getPriezviko (MkSt _meno priezvisko _rocnik _poznanka) =
6      priezvisko
7
8  getRocnik :: Student -> Int
9  getRocnik (MkSt _meno _priezvisko rocnik _poznanka) = rocnik
10
11 getPoznamka :: Student -> String
12 getPoznamka (MkSt _meno _priezvisko _rocnik poznanka) = poznanka
13
14 st :: Student
15 st = MkSt "Jan" "Hrach" 4 "OK"
16
17 mp :: String
18 mp = getMeno st ++ " " ++ getPriezviko st -- "Jan Hrach"
```

Záznamy – špeciálna syntax

```
1 data Student
2   = MkSt {meno :: String,
3           priezvisko :: String,
4           rocnik :: Int,
5           poznamka :: String}
```

- **MkSt** – dátový konštruktor, je stále použiteľný ako funkcia, ale podporuje aj špeciálnu syntax, kde nezáleží na poradí položiek:

- **MkSt** {priezvisko = "Hrach", meno = "Jan",
poznamka = "OK", rocnik = 4}

- okrem toho sú ale automaticky definované funkcie:

```
meno      :: Student -> String
priezvisko :: Student -> String
rocnik     :: Student -> Int
poznamka   :: Student -> String
```

Záznamy – automatické odvodenie inštancie Show

```
1 data Student
2   = MkSt { meno :: String
3           , priezvisko :: String
4           , rocnik :: Int
5           , poznamka :: String
6           } deriving Show
```

- `st = MkSt "Jan" "Hrach" 4 "OK"`
- `print st`
`MkSt {meno = "Jan", priezvisko = "Hrach",`
`rocnik = 4, poznamka = "OK"}`

Záznamy – samostatné odvodenie inštancie Show

```
1 data Student
2   = MkSt { meno :: String
3           , priezvisko :: String
4           , rocnik :: Int
5           , poznamka :: String
6           }
```

- `st = MkSt "Jan" "Hrach" 4 "OK"`

- `print st`

No instance for (Show Student) arising from a use of ‘print’

- `:set -XStandaloneDeriving`

- `deriving instance Show Student`

- `print st`

```
MkSt {meno = "Jan", priezvisko = "Hrach",
      rocnik = 4, poznamka = "OK"}
```

Záznamy – syntax pre aktualizáciu

```
1  data Student
2    = MkSt { meno :: String
3             , priezvisko :: String
4             , rocnik :: Int
5             , poznamka :: String
6             } deriving Show
7
8  st :: Student
9  st = MkSt "Jan" "Hrach" 4 "OK"
10
11 st2 :: Student
12 st2 = st {poznamka = "KO"}
```

- `st2` a `st` sú rôzne záznamy (`st` sa nezmenil)
- `print st2`

```
MkSt {meno = "Jan", priezvisko = "Hrach",
      rocnik = 4, poznamka = "KO"}
```


Union (variants, sum type) – vymenovaný typ

```
1 data Bool = False | True
```

- **Bool** je nový typ
- **False** a **True** sú nové dátové konštruktory bez argumentov
- tento špeciálny prípad variantného typu nazývame aj **vymenovaný typ** (enumerated type)

Union (variants, sum type) – polymorfné typy

```
1 data Maybe a = Nothing | Just a
```

- **Maybe** je nový typový konštruktor
 - $:k \text{ Maybe} \rightsquigarrow \text{Maybe} :: * \rightarrow *$
- **Nothing** a **Just** sú nové dátové konštruktory
 - **Nothing** $:: \text{Maybe } a$
 - **Just** $:: a \rightarrow \text{Maybe } a$

$:kind <typ>$, skrátene $:k <typ>$, ukáže druh typu $<typ>$

- $*$ predstavuje jeden konkrétny typ (napr. **Int**)
- $* \rightarrow *$: typový konštruktor (berie typ a vracia nový typ)
napr. **Maybe** berie **Int** a vracia **Maybe Int**

Definícia

Pokiaľ je dátový typ parametrizovaný typovým parametrom, tak hovoríme, že je **polymorfný**. (Platí pre záznamy, aj var. typy.)

Všeobecný variantný typ pre dve možnosti

```
1 data Either a b = Left a | Right b
```

- **Either** je nový typový konštruktor
 - **:k Either**
 - **Either :: * -> * -> ***
- **Left** a **Right** sú nové dátové konšuktory
 - **Left :: a -> Either a b**
 - **Right :: b -> Either a b**

Rekurzívne typy

rek. def.

```
1 data List a = Empty
2           | Cons a (List a)
3           deriving (Show, Eq, Ord)
4
5 zoznam = Cons 'a' (Cons 'h' (Cons 'o' (Cons 'j' Empty)))
6
7 dump :: List Char -> String
8 dump Empty = ""
9 dump (Cons c list) = c : dump list
```

`dump zoznam` $\rightsquigarrow$ `"ahoj"`